Informe del Laboratorio 1

Funcionamiento y ventajas de los patrones de diseño de agentes de inteligencia artificial

Asignatura: Física Computacional Avanzada

Tema: Patrones de reflexión y uso de herramientas

Entorno de trabajo: Python, Jupyter Notebook y Ollama

1. Introducción

Cuando se habla de agentes de inteligencia artificial, es fácil pensar únicamente en un programa que responde preguntas. Sin embargo, lo interesante de estos sistemas es que pueden organizar su trabajo en varios pasos: producir una respuesta, comprobarla o utilizar funciones externas para resolver una tarea. La manera de ordenar esos pasos se conoce como patrón de diseño.

En los archivos de este laboratorio se presentan dos formas de trabajar con agentes: el patrón de reflexión y el patrón de uso de herramientas. Los dos se apoyan en un modelo de lenguaje que funciona localmente con Ollama, pero persiguen objetivos distintos. El primero intenta mejorar una explicación mediante revisiones; el segundo conecta al modelo con funciones de Python para analizar datos experimentales.

El objetivo de este informe es explicar cómo funciona cada uno, qué ventajas ofrece y cuáles son sus limitaciones, especialmente cuando se usan en física computacional.

2. Patrón de reflexión

2.1. ¿En qué consiste?

La idea del patrón de reflexión es bastante cercana a lo que hacemos al escribir un informe: elaboramos una primera versión, la releemos, encontramos aspectos que se pueden mejorar y hacemos correcciones. En este caso, esas etapas se realizan con ayuda de un modelo de lenguaje.

El agente no entrega necesariamente su primera explicación. Antes puede pedir una revisión que busque errores conceptuales, imprecisiones o falta de claridad. Si la explicación no es aprobada, el sistema genera otra versión a partir de los comentarios recibidos.

2.2. ¿Cómo funciona en el código?

En reflection_agent.py aparecen tres funciones principales. explicar() produce una explicación inicial; revisar() evalúa esa explicación y señala posibles problemas; y corregir() escribe una nueva versión teniendo en cuenta la crítica. La función run_reflection() coordina el ciclo, guarda el registro de cada ronda y se detiene cuando el revisor aprueba la respuesta o se

alcanza el número máximo de revisiones.

Secuencia: pregunta → explicación inicial → revisión → aprobación o corrección → respuesta final.

Un detalle importante es que el explicador y el revisor no son necesariamente dos modelos distintos. En el laboratorio, el mismo modelo puede desempeñar ambos papeles porque recibe instrucciones diferentes en cada etapa.

2.3. Ejemplo aplicado a física

Imaginemos que se le pregunta al agente por qué el período de un péndulo simple no depende de su masa. Una primera respuesta podría decir solamente que la masa no influye. Aunque la idea es correcta dentro del modelo ideal, queda incompleta. El revisor podría pedir que se expliquen las condiciones bajo las cuales ocurre esto.

La versión mejorada puede señalar que, para pequeñas oscilaciones y despreciando la resistencia del aire, el período está dado por $T = 2\pi\sqrt{(L/g)}$. En esa expresión aparecen la longitud del péndulo y la gravedad, pero no su masa. Así, la respuesta deja de ser una afirmación aislada y pasa a incluir el fundamento físico y las aproximaciones utilizadas.

2.4. Ventajas y limitaciones

• Puede mejorar la claridad de una explicación antes de presentarla.

• Permite detectar algunas contradicciones, omisiones o errores conceptuales.

• Se puede adaptar el criterio de revisión según el objetivo: rigor físico, nivel del estudiante o calidad de la redacción.

• Deja un registro de las distintas versiones, lo que resulta útil para estudiar cómo cambió la respuesta.

Su principal limitación es que una revisión automática no equivale a una comprobación independiente. El revisor también puede equivocarse, pasar por alto un error o rechazar una explicación correcta. Además, cada ronda necesita tiempo de cómputo. Por eso, cuando hay resultados científicos o fórmulas importantes, sigue siendo necesario verificarlos por otros medios.

3. Patrón de uso de herramientas (tool calling)

3.1. ¿En qué consiste?

Este patrón parte de una necesidad diferente: un modelo de lenguaje puede entender lo que pide el usuario, pero no conviene confiarle directamente todos los cálculos. En cambio, podemos ofrecerle funciones de Python que realizan tareas específicas y dejar que decida cuándo utilizarlas.

Por ejemplo, si alguien solicita ajustar una curva experimental, el agente interpreta la petición y elige la función adecuada. El cálculo lo realiza una biblioteca científica, no el modelo de

lenguaje por sí solo. Después, el modelo recibe el resultado y lo explica.

3.2. ¿Cómo funciona en el laboratorio?

El archivo agent.py implementa el ciclo de interacción con Ollama. Primero envía al modelo la petición del usuario y la descripción de las herramientas disponibles. Si el modelo solicita una función, el programa identifica su nombre, interpreta los argumentos y ejecuta la función de Python. Finalmente, devuelve el resultado al modelo para que pueda continuar o elaborar una respuesta.

Secuencia: petición → selección de herramienta → ejecución en Python → resultado → interpretación y respuesta.

Esto permite distinguir dos responsabilidades: el modelo decide qué operación necesita y el código se encarga de ejecutarla. La distinción es importante porque la decisión del modelo puede ser incorrecta, pero una función científica bien implementada ofrece un procedimiento numérico explícito y reproducible.

3.3. Herramientas disponibles

Función	Qué hace
list_datasets()	Muestra los conjuntos de datos disponibles.
preview_dataset()	Permite observar algunas filas del conjunto de datos.
list_models()	Enumera los modelos matemáticos de ajuste.
fit_dataset()	Ajusta una curva y calcula parámetros e incertidumbres.
plot_dataset_fit()	Guarda una gráfica con datos, ajuste y residuos.

Los ejemplos del proyecto corresponden a tres experimentos simulados: un resorte, la descarga de un circuito RC y un péndulo simple. Para ellos se contemplan modelos lineales, exponenciales y de potencia.

3.4. Ejemplo aplicado a física

Supongamos que el usuario pide: “Ajusta los datos de un resorte y encuentra su constante elástica”. El agente puede llamar a fit_dataset() con el conjunto de datos del resorte y un modelo lineal. La relación ideal de la ley de Hooke es $F = kx$, mientras que el ajuste implementado utiliza la forma general $F = ax + b$. En ese caso, la pendiente a permite estimar la constante elástica y b representa un posible desplazamiento respecto del origen.

La función utiliza scipy.optimize.curve_fit para obtener los parámetros ajustados y sus incertidumbres. Después, otra herramienta puede producir una gráfica con los puntos medidos, la curva y los residuos. El modelo de lenguaje se encarga de presentar e interpretar esos resultados, pero los números provienen del cálculo de Python.

3.5. Ventajas y limitaciones

• Facilita el uso de cálculos científicos a partir de instrucciones escritas en lenguaje natural.

• Aprovecha bibliotecas como NumPy y SciPy, en lugar de pedir al modelo que invente o estime resultados numéricos.

• Permite añadir nuevas herramientas sin rehacer toda la lógica del agente.

• Ayuda a automatizar tareas repetitivas, como inspeccionar datos, ajustar modelos y generar gráficos.

• Conserva una separación clara entre la interpretación de la solicitud y la ejecución computacional.

La principal dificultad es que el agente puede escoger una función que no corresponde al problema o pasar argumentos incorrectos. También puede interpretar mal una salida numérica aunque el cálculo sea correcto. Por esta razón, es importante revisar el modelo físico elegido, validar los datos y comprobar la coherencia de los resultados.

4. Comparación de los dos patrones

Aspecto	Reflexión	Uso de herramientas
Propósito	Mejorar una explicación.	Realizar operaciones externas.
Proceso	Generar, revisar y corregir.	Elegir, ejecutar e interpretar.
Fortaleza	Revisión de claridad y coherencia.	Cálculos explícitos con Python.
Riesgo	El revisor también puede fallar.	Puede elegir mal la herramienta.
En física	Explicar conceptos.	Analizar datos experimentales.

En mi opinión, lo más interesante es que no se trata de patrones que compitan entre sí. Podrían combinarse: primero, el agente usa una herramienta para ajustar datos y, después, aplica una revisión crítica a la explicación de los resultados. Eso no reemplaza el criterio científico, pero sí puede ayudar a organizar mejor el trabajo.

5. Conclusiones

El laboratorio muestra que un agente de inteligencia artificial puede hacer más que responder de manera inmediata. Según el patrón utilizado, puede revisar su propio texto o apoyarse en funciones externas para realizar tareas concretas.

El patrón de reflexión resulta útil cuando lo más importante es la calidad de una explicación. Su valor está en la posibilidad de identificar fallas y producir una versión mejorada, aunque no asegura por sí mismo que todo sea correcto. El patrón de uso de herramientas, en cambio, es especialmente valioso en física computacional porque permite realizar ajustes y generar resultados mediante programas científicos.

La principal enseñanza es que el diseño del agente debe responder a la naturaleza del problema. Si se necesita explicar un concepto, la revisión puede aportar claridad; si se necesita calcular, conviene recurrir a herramientas verificables. En ambos casos, el análisis

humano sigue siendo indispensable para evaluar si la respuesta tiene sentido físico.

6. Fuentes consultadas

• Material del Laboratorio 1: README.md y reflexion_agent_lab.ipynb.

• Código del patrón de reflexión: reflection_agent.py y display_helpers.py.

• Material del agente de física: README.md y physics_agent_lab.ipynb.

• Código del agente y herramientas: agent.py, physics_tools.py y display_functions.py.

Nota: este informe describe el funcionamiento previsto en el código entregado. No se presentan mediciones ni resultados de ejecución de los notebooks porque no se realizaron pruebas experimentales para este documento.